

Watchtower: Product-Level Architecture for Blockchain Monitoring, Automation, and Governed AI Operations

BIFROST FOUNDATION

Watchtower is a Rust-based, multi-chain monitoring and automation platform for blockchain operations. It executes scheduled rules written in a domain-specific language (DSL), binds those rules to database-managed chain Remote Procedure Call (RPC) endpoints, smart contract calls, contract events, off-chain Application Programming Interfaces (APIs), notifications, prompt templates, webhooks, and AI-agent tasks, and records execution state for auditability. This report presents Watchtower as a product-level technical system rather than as a collection of monitoring scripts. We identify the operational problem that motivates Watchtower, describe the DSL and runtime architecture, introduce the Watchtower Autonomous System (WAS), and explain how AI is applied through two controlled surfaces: Model Context Protocol (MCP)-assisted rule authoring and permissioned runtime offloading to Dobby, a Solidity-oriented AI agent. The central thesis is that blockchain operations require a constrained autonomous system: expressive enough to correlate on-chain and off-chain state, yet governed enough to avoid arbitrary code execution, unapproved external actions, or blocking AI inference in the monitoring loop.

Additional Key Words and Phrases: blockchain infrastructure, automation systems, applied artificial intelligence

1 Introduction

Blockchain infrastructure has moved from single-chain applications to multi-chain products, bridge protocols, oracle-dependent services, vaults, lending systems, exchange integrations, and operational processes distributed across on-chain and off-chain components. This shift creates an observability and automation problem. A safety-critical event may involve a smart contract state change on one network, a transaction receipt on another, a price from an off-chain exchange, a stored historical threshold, and an internal escalation workflow. Traditional monitoring scripts can handle individual checks, but they do not provide a maintainable execution model for this cross-domain behavior.

Watchtower addresses this problem by treating monitoring rules as first-class operational programs. A rule is not merely a cron command; it is a database object with source code, schedule, configuration, execution history, status, storage, permissions, and optional rule-scoped extension keywords. The rule source is written in a purpose-built DSL that includes familiar control flow and blockchain-specific operations. The runtime parses and evaluates this DSL under a controlled interpreter, using database-managed definitions for chains, contracts, calls, APIs, notifications, webhooks, prompts, and AI agents.

The engineering goal is not to create the most general programming environment. The goal is to create a safe operational substrate: expressive enough for real blockchain monitoring, constrained enough for AI-assisted rule generation and production governance, and robust enough to run continuously.

Contributions. This report contributes a product-level explanation of why Watchtower is needed for multi-chain blockchain operations and how its components work together as a coherent system. It first frames the operational gap that appears when teams move from isolated monitoring scripts to persistent monitoring programs that must observe chains, contracts, off-chain APIs, and internal response channels at the same time. It then describes the Watchtower DSL as a controlled operational language, including its syntax categories, value model, and rule-scoped extension mechanisms.

The report also presents the runtime and data architecture that make scheduled, isolated rule execution possible. In particular, it explains why Watchtower's database is more than storage: it is the catalog through which the DSL resolves chains, RPC calls, contracts, APIs, prompts, webhooks, and agent tasks. Finally, the report formalizes the Watchtower Autonomous System (WAS) as a governed model for autonomous monitoring and response

workflows, and it explains why AI should enter Watchtower through constrained MCP-assisted authoring and asynchronous runtime offloading rather than through an unconstrained agent loop.

Terminology. Several terms are used throughout the report. RPC means Remote Procedure Call, the interface used to read chain data such as blocks, transactions, receipts, and logs. ABI means Application Binary Interface, the metadata that describes how smart contract calls and events are encoded. AST means Abstract Syntax Tree, the typed internal representation produced after the DSL parser reads a rule. MCP means Model Context Protocol, the tool interface through which AI assistants read documentation, verify rules, run dry-runs, and submit structured mutations. WAS means Watchtower Autonomous System, the governed operating model that combines rules, extensions, execution state, permissions, and AI-assisted analysis. OAuth context refers to the authenticated user or client metadata attached to management actions.

The main runtime modules are the **Service**, which initializes shared state; the **Spawner**, which discovers enabled rules; the **Watcher**, which schedules and runs one rule; the DSL interpreter, which evaluates parsed rule programs; and the dispatcher, which routes approved domain operations to chain, API, notification, webhook, prompt, and agent handlers. The validation engine records execution outcomes, errors, and status transitions. Dobby refers to the permissioned AI-analysis agent used by Watchtower for runtime analysis tasks. An offloaded task is a persisted AI job that runs outside the scheduler path; ReplyTo and AndThen describe how the asynchronous result is delivered back to a message thread or follow-up callback.

2 Problem Statement

Watchtower is motivated by a concrete operational gap: blockchain monitoring has to combine chain access, contract semantics, off-chain context, durable state, and governed response actions in one execution model. This section first states the requirements that such a platform must satisfy, then explains why ordinary cron scripts and hand-written bots do not scale to that role.

Operational Requirements. A production blockchain monitoring platform must satisfy requirements that are not met by generic infrastructure monitoring. The first requirement is multi-chain reachability. A rule must be able to call RPC methods, read block and transaction data, decode logs, and query contract state across networks whose RPC surfaces and response formats are not identical. Treating these interactions as ordinary HTTP calls would push too much chain-specific parsing logic into each rule, so Watchtower has to provide blockchain-aware bindings at the language level.

The second requirement is contract awareness. Operators think in terms of oracles, vaults, sockets, bridges, and protocol modules, but the underlying system needs addresses, ABIs, function selectors, event signatures, default parameters, and response paths. Watchtower therefore has to bind human-readable operational names to contract metadata. A related requirement is off-chain correlation: practical monitoring often compares on-chain state with prices, exchange data, protocol APIs, or internal services. A blockchain rule engine that cannot integrate off-chain data will miss important operational context.

The system must also be stateful. Many useful alerts depend on what happened in the previous execution: the last processed block, the last observed price, the current alert state, or the previous execution status. This state must be rule-scoped and persistent, because losing it can produce duplicate alerts or missed events. At the same time, response actions must be governed. Watchtower rules should be able to notify humans, call approved external services, and request AI analysis, but they should not receive arbitrary network or code execution authority. Finally, the system must run continuously, recover from ordinary failures, record its executions, and remain structured enough for AI assistants to generate and validate rules without crossing operational boundaries.

Why Scripts Are Insufficient. Cron scripts and hand-written bots are attractive at the start of a protocol project because they are simple to write. They become a liability once the number of monitored systems grows. Scripts

duplicate RPC handling, response parsing, error recovery, alert formatting, state persistence, and permissions. They are also difficult to delegate to AI tools because a generated script can perform arbitrary computation or network access. Watchtower replaces this pattern with a constrained language and a database-managed execution model.

Table 1. Design contrast between ad hoc scripts and Watchtower.

Concern	Ad hoc scripts	Watchtower
Execution	Process-level scripts, inconsistent lifecycle	Rule-level watchers with scheduler and restart behavior
State	Local files or custom tables	Rule-scoped storage and execution records
Extensions	Code changes per integration	Database-driven RPC, API, contract, webhook, prompt, and agent definitions
AI generation	Unsafe unless sandboxed externally	DSL grammar, MCP schemas, verification, dry-run
Governance	Usually implicit	Rule permissions and admin approval flows

3 Design Principles

Watchtower follows a small set of design principles that are easier to understand together than as separate feature claims. The first principle is constrained expressiveness. The DSL includes variables, control flow, arrays, objects, property access, and domain operations, but it intentionally avoids arbitrary system access. A rule can express meaningful blockchain monitoring logic without becoming an unrestricted script. This distinction matters because the runtime, not the rule author or an AI model, remains responsible for deciding which effects can occur.

The second principle is database-driven extensibility. The grammar does not need to change every time a new chain, contract, API, prompt, or agent task is added. Instead, the runtime resolves many DSL keywords through database entities. A chain RPC call, contract function, off-chain API, webhook, prompt, or agent task can be registered, reviewed, and reused as operational vocabulary while the DSL itself remains stable. This is the main mechanism that lets Watchtower grow as a product rather than as a collection of hard-coded monitors.

The third principle is runtime isolation. Rules are executed by independent watchers, and watcher lifecycle is tied to rule state. A disabled or deleted rule exits permanently; recoverable failures can be restarted; source changes are detected and the interpreter is rotated without restarting the whole service. Isolation makes failure handling local. A broken rule should create a recorded execution failure for that rule, not a service-wide outage.

The final principle is governed autonomy. Watchtower allows autonomous workflows, but those workflows run inside explicit permission and approval boundaries. Webhooks and AI agents require approval paths and permission records, and long-running AI tasks are offloaded so inference does not block the rule scheduler. The system can monitor, decide, notify, and request analysis without turning every rule into an unconstrained agent.

4 System Overview

Watchtower is implemented as a Rust workspace and a companion MCP server. The runtime consists of the application layer, DSL engine, data layer, and shared primitive configuration. The MCP server exposes documentation, verification, dry-run, query, mutation, and admin-request tools to AI assistants and other clients.

The application layer is responsible for lifecycle and orchestration. The **Service** builds the shared interpreter context, initializes database vacuuming, creates the validation engine, and starts the **Spawner**. The **Spawner** scans enabled rules, parses each DSL script, selects the active test or business schedule from rule configuration, and launches one **Watcher** per active rule. Each **Watcher** waits for the next cron trigger, verifies that the rule

still exists and remains enabled, reloads updated source code, executes the interpreter, and records execution results. This structure makes scheduling explicit while still allowing each rule to run with its own lifecycle.

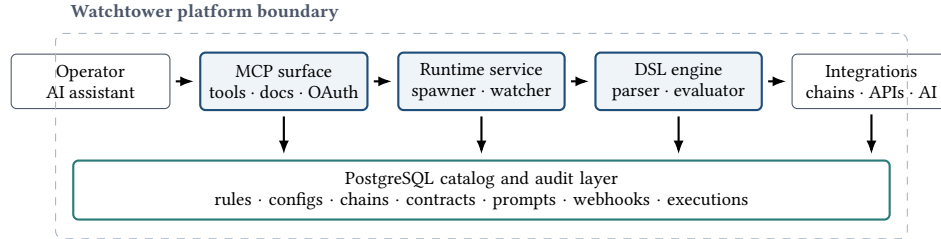


Fig. 1. High-level Watchtower architecture. The database is not only persistence; it is part of the language extension model.

The DSL engine is the deterministic execution core. It is divided into parser, interpreter, evaluator, dispatcher, and conversion utilities. The grammar is defined in `dsl.pest`, and the parser builds typed Abstract Syntax Tree (AST) values from source. The evaluator recursively processes statements and expressions, while the function caller dispatches controlled external operations such as chain RPCs, contract calls, off-chain APIs, notifications, and agent tasks. System operators implement storage, logging, conversion, JSON utilities, webhooks, and prompt operations. The result is a runtime where external effects are visible in the language and routed through known dispatch points.

The data layer uses SeaORM, an object-relational mapper, over PostgreSQL. It contains ordinary runtime state, but it also stores the operational vocabulary used by the DSL. For example, `chain_rpc_calls` maps a DSL keyword to an RPC method, default parameters, DSL parameter positions, and response extraction paths. The same pattern appears in contract calls, off-chain API calls, webhooks, prompts, and agent tasks. This is why the database is shown as part of the language extension model in Figure 1: it is the catalog through which a stable DSL can interact with changing infrastructure.

The MCP server provides the AI-compatible management surface around this runtime. It exposes syntax guides, examples, grammar, rule queries, rule creation, verification, dry-run, configuration update, and admin request tools. This is a product-critical boundary. AI assistance is useful only when the system gives the model a bounded interface with structured inputs, validation, and auditable mutation paths.

5 Data Model and Extension Catalog

Watchtower’s database schema is a product feature, not an implementation detail. The schema encodes which operational effects are allowed and how the DSL resolves names at runtime. This design lets Watchtower evolve from a fixed set of hard-coded monitors into a platform where operational vocabulary is registered, reviewed, reused, and audited.

Core Entity Groups. The data model can be divided into five entity groups. Rule entities represent source code, scheduling, thresholds, history, execution state, and audit output. These records are the durable operational identity of a rule: they make it possible to ask what code ran, when it ran, how it was configured, and what result it produced.

The blockchain catalog transforms chain-specific details into stable DSL keywords for networks, RPC endpoints, contract calls, and contract events. The off-chain catalog performs a similar role for URL templates, methods, parameters, headers, pagination, and response extraction. Together, the on-chain and off-chain catalogs let rules refer to operational capabilities without embedding provider-specific wiring in every script.

The autonomy catalog adds rule-scoped extension points for dynamic contract bindings, approved webhook actions, and reusable prompt templates. The agent catalog represents external AI agents, callable tasks, permission boundaries, sessions, and asynchronous execution state. This split keeps deterministic monitoring, external action, and AI execution visible as separate governed surfaces.

The main rule-scoped extension records are Rule Contract Keywords, Rule Webhook Keywords, and Rule Prompt Keywords. A Rule Contract Keyword (RCK) binds a rule-local name to contract metadata such as chain, address, and ABI. A Rule Webhook Keyword (RWK) binds a rule-local name to an approved external HTTP action. A Rule Prompt Keyword (RPK) binds a rule-local name to a parameterized prompt template used for AI-assisted analysis. These records let a rule gain task-specific vocabulary without embedding addresses, URLs, or long prompt text directly in the DSL source.

Response Extraction as a Language Feature. Many monitoring systems return raw JSON or force each script to parse responses manually. Watchtower moves response extraction into database metadata. RPC and API definitions can define default parameters, DSL parameter mapping, and response paths. The interpreter can therefore return DSL-level values rather than exposing every rule to heterogeneous provider responses. This is one of the key reasons the DSL remains concise even when the underlying chains and APIs differ.

Versioning and Operational Memory. Rules are not ephemeral. They have committed versions, configuration records, status records, execution records, logs, and storage. This creates operational memory. A rule can remember the last processed block, last observed price, previous alert state, or previous execution result, while the platform can audit what source code and configuration produced an alert. This distinction is important for technical positioning: Watchtower is not only a DSL interpreter; it is an operational data system around the interpreter.

Table 2. Database-backed extension surfaces exposed to the DSL.

Surface	Example DSL form	Product role
Chain RPC	balance, block, transaction calls	Standardizes chain access and response extraction while allowing RPC metadata to be managed outside source code.
Contract call	ABI-backed function calls	Makes ABI-backed contract reads available as named operational functions.
Event decode	ABI-backed log decoding	Converts raw topics and data into structured rule values.
Off-chain API	price, exchange, product APIs	Correlates chain observations with market, product, or service data.
RCK	dynamic contract keyword	Adds rule-scoped contract integration without global grammar changes.
RWK	approved webhook keyword	Turns monitoring conditions into approved external actions.
RPK	parameterized prompt keyword	Converts runtime data into reusable prompt payloads.
Agent task	Dobby session and prompt task	Offloads specialized AI analysis under permission control.

6 Watchtower DSL

The DSL is Watchtower’s control surface for expressing monitoring logic without granting arbitrary program authority. This section describes the program shape, the domain operations exposed to rules, and the rule-scoped extension keywords that let product teams bind dynamic contracts, webhooks, and prompts to a specific monitoring workflow.

Program Structure. The DSL program is a sequence of statements. The statement model includes assignments, object-property assignments, array-element assignments, expression statements, conditionals, loops, and control-flow statements.

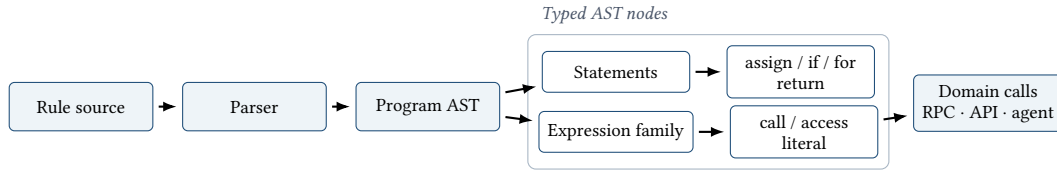


Fig. 2. Simplified DSL Abstract Syntax Tree organization.

Expressions follow a conventional precedence hierarchy: ternary expressions, logical operations, negation, comparisons, additive expressions, multiplicative expressions, and primary values. Primary values include booleans, numbers, strings, addresses, arrays, objects, function calls, contract references, contract events, agent sessions, array access, property access, chain identifiers, and null.

Domain Operations. The language is organized around operational function families. Chain RPC calls cover balance, block, transaction, receipt, and event queries, while contract calls operate against pre-defined database contracts. Dynamic contract calls use Rule Contract Keywords (RCK), allowing a rule to bind a contract address and ABI for a specific monitoring purpose without changing the global language. ABI-backed event definitions provide event decoding, converting raw log topics and data into values that the interpreter can compare, store, and pass to other operations.

Off-chain API calls use configured URL, path, query, header, pagination, and response extraction metadata. System operations provide storage, logging, time, math, hex, string, and JSON handling. Notification operations deliver alerts to Slack, and autonomous workflow operations connect rules to approved webhooks, prompt templates, and agent tasks. The result is a language surface that feels specific to blockchain operations while still being narrow enough for validation, dry-run, and permission checks.

The following fragment is illustrative. Bifrost is used as an example chain namespace, DM denotes a direct-message notification channel, and the prompt name shows how an RPK can be referenced from DSL code without embedding the full prompt template in the rule.

```

from_block = Storage.Get("last_bridge_block");
to_block = Bifrost.LatestBlock();

events = Bifrost.GetEvents(
    Bifrost.Socket,
    [Bifrost.Socket.Socket],
    from_block,
    to_block
);

for event in events {
    decoded = Bifrost.Socket.Socket.DecodeEvent(
        event.topics,
        event.data
    );

    if decoded.amount > 100000 {
        ts = Slack(DM, "High-value bridge event: " + event.tx_hash);
        prompt = Prompt.Build("RPK:bridge-risk-review",
            [event.tx_hash, decoded.amount, decoded.sender]);

        session = Dobby.OpenChildSession(Dobby.session_CCCP);
        if session.status {
            sid = session.result.id;
            Dobby.Prompt(sid, {
                "parts": [{"type": "text", "text": prompt}],
                "agent": "general"
            })
        }
    }
}

```

```

    }).ReplyTo(DM, ts);
  }
}
}

Storage.Set("last_bridge_block", to_block);

```

Listing 1. A representative Watchtower DSL rule fragment.

Rule-Scoped Extension Keywords. Watchtower defines three rule-scoped extension mechanisms that are central to WAS. Rule Contract Keywords (RCKs) provide dynamic contract bindings. An RCK contains a rule-scoped keyword, chain, address, and ABI, which means a rule can interact with a contract that is relevant to a specific monitoring task without promoting that contract into the global built-in catalog. This is important for product operations because new contracts, test deployments, or incident-specific contracts can be monitored quickly while still remaining explicit database records.

Rule Webhook Keywords (RWKs) provide controlled external HTTP actions. They are appropriate for actions such as creating tickets, routing incidents, updating internal dashboards, or notifying another service that a rule has detected a condition. The key design point is that the webhook is not an arbitrary URL hidden inside the DSL source. It is a named, reviewable extension associated with a rule and subject to an approval flow.

Rule Prompt Keywords (RPKs) provide parameterized prompt templates. They let a rule transform runtime observations into a reusable AI prompt payload without hard-coding long prompts in every script. This also makes prompt behavior more consistent: the same operational template can be reviewed, reused, and invoked with different transaction hashes, addresses, amounts, or rule identifiers.

7 Watchtower Autonomous System

We define the Watchtower Autonomous System (WAS) as the governed autonomous operating layer produced by combining DSL execution, persistent state, rule-scoped extensions, notification, webhooks, prompt templates, AI-agent offloading, and audit records.

WAS Model. WAS can be understood as a layered operating model. At the top, a human operator or an AI assistant expresses an operational intent through the MCP interface. That intent is then made concrete as a rule: DSL source, schedule, thresholds, reporting behavior, and purpose metadata. This rule layer is where an operational requirement becomes an executable artifact.

The rule is extended through RCK, RWK, and RPK records. These records connect the rule to dynamic contracts, approved external actions, and reusable prompt templates without making those capabilities implicit in the source code. The execution layer then runs the rule through the spawner, watcher, interpreter, and validation engine. It is responsible for scheduling, evaluating statements, calling external systems through controlled dispatchers, mutating rule-scoped storage, and recording execution results.

Above the deterministic execution layer is the intelligence layer. Prompt operations and Dobby sessions add AI-assisted analysis, but the AI model does not become the scheduler or the owner of rule execution. The governance layer wraps the entire system through permissions, admin approvals, user context, logs, statuses, and offloaded task records. This layered structure is why WAS can support autonomous workflows without becoming an unconstrained autonomous agent.

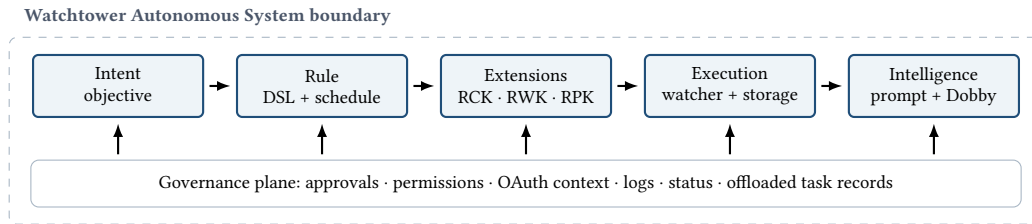


Fig. 3. Layered model of the Watchtower Autonomous System. WAS is autonomous only inside a governed rule-execution boundary.

Autonomy Without Arbitrary Agency. WAS is deliberately not a free-form agent. A free-form agent decides what tools to call at runtime. WAS instead executes pre-validated DSL logic where each external operation is represented by an approved keyword or database entity. The system can behave autonomously in the operational sense—monitor, decide, notify, ask for analysis, and trigger workflows—without giving the language unbounded authority.

8 Applying AI to Watchtower

AI is applied at two deliberately separate surfaces: authoring time and runtime. The distinction is central to the product design. Authoring-time AI helps operators produce correct DSL source and supporting extension records; runtime AI helps an already approved rule enrich an alert or incident workflow with specialized analysis. Watchtower does not let either surface become an unbounded agent loop.

At authoring time, the AI assistant works through MCP tools. It can read the DSL reference, examples, and integration guides, then draft a rule and any supporting RCK, RWK, or RPK definitions. The next steps are not informal: the assistant should call the verifier, dry-run the rule with bounded execution, and only then submit a structured create or update request. This workflow reflects the code boundary in the MCP server: documentation tools supply canonical syntax, verifier tools parse and exercise source, modifier tools persist state through typed schemas, and admin-request tools separate rule creation from sensitive capability approval.

At runtime, Dobby AI operations are DSL calls rather than arbitrary model decisions. The interpreter resolves the agent and task, builds the request, checks whether the agent is enabled, checks whether the current rule has permission to invoke that agent, and then either executes the call or persists an offloaded task. Offloading is important because model inference is slower and less predictable than deterministic rule evaluation. A scheduled watcher should keep monitoring even if AI analysis takes longer than expected. The offloaded task record, callback snapshot, and reply target preserve enough state for asynchronous completion without blocking the scheduler. In this model, ReplyTo routes an AI result back to the original alert thread, while AndThen represents a follow-up callback that resumes with saved interpreter context after the AI task completes.

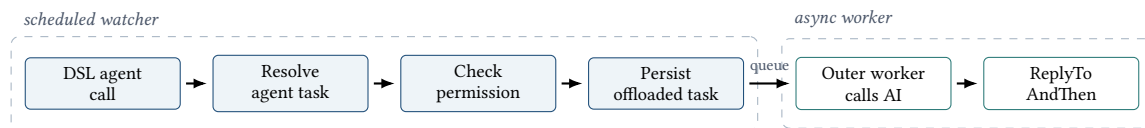


Fig. 4. Runtime AI-agent execution path.

This AI model is correct for Watchtower because the system assigns authority to the deterministic runtime and value to the AI assistant. The runtime owns scheduling, permission checks, storage mutation, execution records,

and delivery semantics. The AI assistant contributes rule drafting, incident analysis, summary generation, and security-review payloads. That split allows AI to improve operator leverage while avoiding the two failure modes that would make monitoring unsafe: unapproved external action and blocking inference in the rule loop.

9 MCP Tool Surface

The MCP server is the product boundary between AI assistants and Watchtower state. Its purpose is not only convenience. It converts informal operational intent into schema-constrained tool calls that can be checked, audited, and replayed. This design is essential because language models are effective at drafting rules but should not write directly to production databases or invent undocumented function signatures.

The MCP server groups tools into document tools, verifier tools, query tools, modifier tools, and system/admin tools. These families mirror the lifecycle of a rule. Documentation tools reduce hallucination by giving the assistant the canonical grammar and examples. Verifier tools make syntax and dry-run feedback available before persistence. Query tools let the assistant inspect existing rules, contract keywords, status records, and related catalog entities. Modifier tools create or update rules and extension records through typed schemas. Admin request tools represent a governance boundary for webhooks and AI-agent access.

A robust AI authoring flow follows five steps. First, the assistant retrieves the DSL reference and relevant guides. Second, it drafts the rule and any supporting RCK, RWK, or RPK definitions. Third, it runs syntax verification. Fourth, it executes a dry-run with bounded timeout and ephemeral dynamic contract data when needed. Fifth, it creates the rule and requests admin approval for sensitive capabilities. This sequence makes the assistant useful while keeping the production system in charge of validation and authorization.

Without MCP, AI integration tends to be prompt-only: an operator asks a model for code, copies it into a system, and hopes the code is correct. Watchtower’s MCP layer changes that interaction into a controlled protocol. The model can read canonical documentation, call a verifier, call a dry-run tool, and submit a structured create request. The system can then reject malformed input, preserve creator context, and route approval requests to administrators. This is a quality improvement rather than a cosmetic integration: it turns AI output into auditable product state and gives operators a reproducible path from intent to verified rule.

Table 3. MCP tool families and their role in controlled AI authoring.

Family	Representative tools	Role
Documentation	reference, grammar, examples, guides	Gives an AI assistant a stable source of syntax and examples before it writes a rule.
Verification	parse verification and dry-run	Catches parser errors and execution issues before a rule becomes active.
Queries	rule, contract, keyword, status reads	Supports inspection, debugging, and status reporting.
Modification	rule/config updates and RCK/RWK/RPK tools	Persists rules and dynamic extension records through typed schemas.
Admin requests	webhook and AI-agent approval requests	Separates rule creation from approval of sensitive effects.

10 Threat Model and Assumptions

Watchtower is an operational automation platform. Its security model therefore focuses on preventing unsafe automation, preserving auditability, and limiting the blast radius of incorrect rules or unavailable dependencies. It is not a consensus protocol and does not attempt to prove the correctness of external blockchains, RPC providers, or AI model outputs.

The threat model includes both human and non-human actors. Rule authors create or update DSL rules through MCP or administrative tools, and their mistakes can become production risks if the system does not validate source code and extension references. AI assistants are a second actor class. They can improve authoring speed, but their output must be treated as generated input that requires parsing, dry-run, and permission checks before activation.

Runtime operators form the approval and response layer. They decide whether a rule should receive access to sensitive integrations such as webhooks or AI-agent tasks, and they consume alerts when rules fire. External services form another class of actors: RPC endpoints, off-chain APIs, Slack, webhook targets, and Dobby can all be unavailable, slow, or inconsistent. Finally, Watchtower must account for adversarial or unexpected events, including malformed API responses, unexpected contract logs, compromised webhook targets, and misleading AI outputs.

The platform assumes that PostgreSQL, configured secrets, and the host runtime are operated in a trusted environment. It also assumes that administrators are responsible for granting AI-agent and webhook approval only to rules that have been reviewed. Chain RPC endpoints and off-chain APIs may fail or return unexpected data, so the interpreter and validation engine must record errors rather than silently accepting inconsistent execution.

Watchtower does not attempt to make AI output authoritative. AI output is an analysis artifact delivered to a channel or follow-up callback, not a consensus fact. Watchtower also does not eliminate the need for protocol-level security audits. Its role is continuous observation, automation, and escalation.

The principal risk control is grammar-bounded execution. Watchtower evaluates DSL source with a fixed parser and interpreter rather than executing arbitrary scripts. External capabilities are represented as explicit database records, so a rule's operational reach can be inspected through its referenced chains, contracts, APIs, webhooks, prompts, and agents. Before a rule becomes active, verification and dry-run provide an early check against syntax errors and obvious execution failures.

Sensitive actions are controlled separately from ordinary rule execution. AI-agent calls require rule-specific permissions, and webhook or AI-agent use can trigger admin approval requests. Long-running AI inference is persisted as an offloaded task rather than blocking the watcher loop. If an incident occurs, execution records, logs, rule status, and offloaded task state provide material for post-incident review. These controls are complementary: grammar bounds what the source can express, catalog records bound what integrations exist, permissions bound who can invoke sensitive capabilities, and execution history preserves evidence after the fact.

11 Reliability and Safety

Reliability in Watchtower is built from small operational boundaries rather than from a single global supervisor assumption. This section explains how watcher isolation, explicit governance records, and safety invariants work together so that rule failures, sensitive integrations, and AI operations remain observable and constrained.

Watchtower's reliability model begins with isolated watcher tasks. Each enabled rule is assigned its own watcher, so a failure in one rule does not require the entire service to stop. Watchers use cron-based scheduling, and they reload schedule state when rule configuration changes. When a rule is deleted or disabled, its watcher exits permanently; when a recoverable failure occurs, the service can restart the watcher after a delay.

External calls are treated as fallible operations. Chain RPC and off-chain API calls include retry behavior, while execution outcomes and errors are recorded through the validation engine. Database vacuuming provides retention control and operational hygiene for accumulated execution records. Sentry integration, used here as an external error-monitoring sink, adds visibility when runtime failures escape normal rule-level handling.

The governance model is centered on explicit representation. Rules, configs, commits, contract keywords, webhook keywords, prompt keywords, agent permissions, executions, and offloaded tasks are database records.

This makes the system inspectable. Sensitive actions such as webhooks and AI-agent access have admin approval request flows, and AI-agent execution checks `agent_rule_permissions` before performing the task.

The safety argument can be summarized through a small set of invariants. First, rule execution is not arbitrary execution: DSL source is parsed by a fixed grammar and evaluated by Watchtower’s interpreter. Second, extension behavior is not invisible. Dynamic resources such as RCKs, RWKs, RPKs, and agent tasks are explicit database entities rather than hidden code paths.

Third, AI operations are not automatically trusted. Runtime agent operations require enabled agents and rule-specific permission records. Fourth, AI inference does not block deterministic monitoring; slow tasks can be offloaded and later resumed through stored task records and callback snapshots. Finally, the platform avoids state ambiguity by persisting rule executions, statuses, logs, storage mutations, and task outcomes. These invariants explain why Watchtower can support autonomous behavior without granting open-ended authority to rules or models.

12 Operational Flows

The preceding sections describe Watchtower’s components; this section follows how those components behave in real workflows. It covers the lifecycle of a rule from authoring to scheduled execution and then walks through a representative bridge-monitoring flow that combines chain reads, event decoding, notification, and asynchronous AI analysis.

A rule begins as an operational objective. The objective may be written by a human operator or drafted with assistance from an AI model, but it becomes actionable only when it is represented as DSL source and associated metadata. Before persistence, the MCP verifier parses the rule and returns syntax feedback. A dry-run can then execute the rule under a bounded timeout using the shared context and, when needed, ephemeral dynamic contract definitions. This allows many mistakes to be caught before a watcher is ever created.

Once the rule is accepted, the creation tool stores the DSL source, rule configuration, and any supporting RCK, RWK, or RPK definitions. The spawner periodically discovers enabled rules and creates watcher tasks for rules that are not already active. Each watcher executes on its configured schedule and records execution results through the validation engine. If the rule source or schedule changes, the watcher rotates its interpreter or schedule state rather than requiring a full service restart.

A representative WAS workflow is bridge threat monitoring. The rule first queries recent bridge events from a configured chain, using persistent storage to determine the last processed block. It then decodes logs with ABI-aware event definitions so that raw topics and event data become structured values. The DSL can filter those values for high-value transfers, unusual counterparties, or other product-specific risk conditions.

When the rule detects a condition worth escalation, it posts an initial Slack alert and captures the thread timestamp returned by the notification call. It then builds an RPK prompt that contains the transaction hash, amount, actor data, and any decoded fields relevant to the review. The rule can open a product-specific Dobby session and offload the prompt so that AI analysis is delivered back to the Slack thread without blocking the scheduled rule. If the workflow requires follow-up automation, an `AndThen` callback can run with the restored interpreter environment after the offloaded task completes.

This flow demonstrates why the product is more than alerting. It combines stateful block progress, ABI-aware decoding, risk filtering, human notification, prompt templating, agent permission checks, asynchronous AI analysis, and auditable execution records. A hand-written script could implement one version of the flow, but Watchtower turns the pattern into governed operational infrastructure that can be authored, reviewed, reused, and inspected.